

Verification Continuum™

IDX Waveform Replay User Guide

Version W-2024.09, September 2024



Copyright and Proprietary Information Notice

© 2024 Synopsys, Inc. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>. All other product or company names may be trademarks of their respective owners.

Free and Open-Source Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

www.synopsys.com

Contents

Customer Support	5
Synopsys Statement on Inclusivity and Diversity	5

1. Introduction	7
------------------------------	----------

2. IDX Waveform Replay Simulation Usage Model	8
Setting Up the Environment	8
Preparing the Inputs	8
The Waveform (FSDB)	9
The simv Executable	9
The Mapping File	9
The Configuration File	10
Running IDX Waveform Replay Simulation	11
simv Command Usage	11
Using IDX in Verdi/VCS Interactive Debugging	12
Checking IDX Results	12
Checking Files After IDX Simulation	12
Sanity Checks Between the Used Waveform and the IDX Results	13
IDX Waveform Replay Simulation Configuration	14
Overview	14
Common Fields	15
Record Fields	16
Examples of the Configuration File	18
IDX With Multiple Waveforms	19
Same or Overlapped Replayed Time Windows	19
Non-overlapped Replayed Time Window	20
Executing an Example	21
The Design	21
SoC Waveform for Reuse	22
Compiling the IP Design	23
Replaying SoC Waveform Stimulus to the IP Design	23
Checking the Results	24
Environment Variables	25

Contents

Limitations and Known Issues 27

Preface

IDX enables reusing waveforms to avoid continually re-creating and re-running stimulus. As the waveform (usually FSDB) records the values, each signal/bus contains throughout the simulation run, it is a good source to 'replay' the values as the stimulus in a simulation.

Customer Support

For any online access to the self-help resources, you can refer to the documentation and searchable knowledge base available in SolvNetPlus.

To obtain support for your Verdi product, choose one of the following:

- Open a case through SolvNetPlus.

Go to <https://solvnetplus.synopsys.com/s/contactsupport> and provide the requested information, including:

- Product L1 as Verdi
- Case Type

Fill in the remaining fields according to your environment and issue.

- Send an e-mail message to support_center@synopsys.com.

Include product name (L1), sub-product name/technology (L2), and product version in your e-mail, so it can be routed correctly.

Your e-mail will be acknowledged by automatic reply and assigned a Case number along with Case reference ID in the subject (ref:____:ref).

For any further communication on this Case via e-mail, send e-mail to support_center@synopsys.com and ensure to have the same Case ref ID in the subject header or else it will open duplicate cases.

Note:

In general, we need to be able to reproduce the problem in order to fix it, so a simple model demonstrating the error is the most effective way for us to identify the bug. If that is not possible, then provide a detailed explanation of the problem along with complete error and corresponding code, if any/permissible.

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary

Contents

language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IPs. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IPs implement industry-standard specifications that are currently under review to remove exclusionary language.

1

Introduction

Waveform (FSDB) records the signals activities in simulation or emulation. These activities are usually a very good source to reuse to accelerate the debug and validation iterations.

IDX (Intelligent Debug Acceleration) is an intelligent waveform reuse tool. It offers the capability to read the signals value changes in a waveform and apply the values to the specified signals in a VCS compiled simulation executable (`simv`). With the reuse of the signal activities in simulation, IDX can avoid continually re-creating and re-running stimulus.

IDX enables many valuable use cases to speed up debug and validation, such as SoC to IP debug, Encrypted IP debug, Assertion Modules Validation, Testbench Interactive Debug Acceleration, etc. The kernel of these use cases is the IDX 'waveform replay' simulation engine. This chapter focuses on the usages of IDX waveform replay simulation.

2

IDX Waveform Replay Simulation Usage Model

The following sections describe how to enable IDX Waveform Replay Simulation flow and gives a demo case to show how it works.

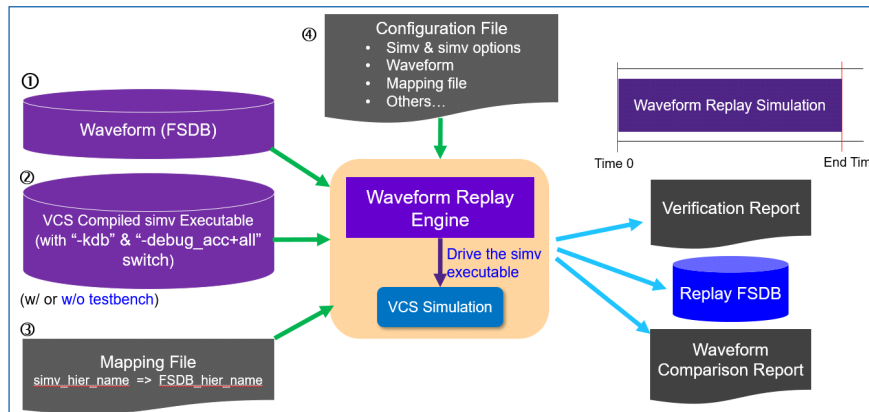
Setting Up the Environment

You need to run the following command in the C (CSH) shell environment before running IDX:

```
% setenv VERDI_HOME <verdi_installation_path>
% setenv PATH $VERDI_HOME/bin:$PATH
```

Preparing the Inputs

The following figure illustrates the flow in IDX waveform replay simulation:



To enable IDX waveform replay simulation, below four data are necessary to be prepared as the inputs:

1. The waveform (FSDB) for reuse
2. The VCS compiled `simv` executable for simulation

3. The mapping file which specifies the signal names to be 'replayed' from the waveform to the `simv`
 4. The IDX configuration file
-

The Waveform (FSDB)

IDX allows to read the waveforms, that is, FSDB generated by any of the tools.

The `simv` Executable

IDX intends to put values to the VCS compiled `simv` executable. Therefore, the following VCS compilation switches must be used:

```
-debug_acc+f+w+cbk+fwn, (or -debug_acc+all)
```

The following options are not mandatory but strongly suggested to add to:

- `-kdb`
- `-deraceclockdata=lerupdate`

The `-deraceclockdata=lerupdate` option is used to resolve the potential race condition issues during in waveform replay simulation.

The Mapping File

IDX reads the mapping file with the following format per line:

- `<simv_signal_full_hierarchical_name> => <waveform_signal_full_hierarchical_name>`
- `<simv_signal_full_hierarchical_name> => <const_value>`

The following is an example of a mapping file used in IDX:

```
// It is OK to use double slash to comment out a line

// LHS (left-hand-side): simv full hierarchical name
// RHS (right-hand-side): waveform full hierarchical name
TEST.top.cpu.clock => TEST.top.cpu.clock

// The hierarchies between LHS and RHS can be independent
TOP.dut.cpu.data => test.u_top.dut.core.data

// Part selection or bit-selection is allowed
TOP.dut.cpu.pmm.addr[3] => test.addr[0]
TOP.dut.cpu.pmm.addr[2:0] => test.addr_1

// Constant values in SystemVerilog format is allowed in RHS
```

```
TEST.top.cpu.reset => 1'b0
TEST.top.cpu.scan_in => 2'b00
```

With the mapping, IDX is going to read the values of RHS from the waveform and put the values to LHS of `simv` during the simulation.

The Configuration File

IDX requires a configuration file to describe the inputs and the options used during the waveform replay simulation. Following is the basic format of the configuration file:

```
#####
# Common fields
# The field here will give impact on all waveform files
#####

//simv.daidir abs. path. [MUST HAVE] (Optional in simv. first usage)
SIMV_DAIDIR = $PWD/simv.daidir

//necessary runtime options for running original simulation. (Optional)
SIMV_OPT = +UVM_VERBOSITY=UVM_HIGH +UVM_TESTNAME=eth_test -l simv_eth.log

//specify the path where the simv executable is going to run. (Optional)
//default is the current directory if not specified
SIMV_RUN_PATH =

//waveform dumping Tcl commands. Use semicolon to separate
tasks (Optional)
WAVEFORM_DUMPING_TASK = fsdbDumpvars +all; fsdbDumpSVA

#####
# Record fields (Waveform dedicated fields)
# The field here will give impact to the waveform file for each record
only #####

##### record 1 #####
{
//waveform file abs. path. [MUST HAVE]
WAVEFORM = $PWD/novas.fsdb

//signal mapping list. [MUST HAVE]
MAP= $PWD/wave_rep.map
}

##### record 2 #####
{
//can be empty as a placeholder
}
```

In general, there are two fields used in the configuration file.

- Common field:
 - This field describes the information about the VCS compiled `simv` executable. The `SIMV_DAIDIR` setting is a must-have.
- Record field:
 - This field describes the information about the waveform and the mapping file.
 - As IDX allows 'replaying' multiple waveforms, it is possible to have multiple records. Each record must describe the information about the waveform and the corresponding mapping file.
 - The record field can be empty as a placeholder; but the entire configuration file must contain at least one record field.

[IDX Waveform Replay Simulation Configuration](#) gives detailed description to the configuration setting. [IDX With Multiple Waveforms](#) describes how to use multiple record fields to run IDX waveform replay simulation.

Running IDX Waveform Replay Simulation

idx Command Usage

IDX uses the "idx" executable for waveform replay simulation. The syntax of "idx" usage is as follows:

```
% idx -cfg <IDX_configuration_file> -work <IDX_working_directory>
```

Where:

- `<IDX_configuration_file>`: the configuration file described in the [Checking IDX Results](#).
- `<IDX_working_directory>`: the directory where the IDX results are present.

simv Command Usage

IDX allows to use the `simv` executable to launch IDX waveform replay simulation. The `simv` runtime options are as follows:

```
% simv -ucli +idx_cfg <IDX_configuration_file> \
+idx_work <IDX_working_dir> <other_simv_options>
```

Where `-ucli` is a **necessary** option to enable IDX on `simv`.

In the `simv` first usage, IDX can identify the `simv` executable and the `simv` options automatically. Therefore, it is **unnecessary** to set the following attributes in the common fields in the configuration file:

- `SIMV_DAIDIR = ...`
- `SIMV_OPT = ...`

If the configuration file still contains the `SIMV_DAIDIR` attribute, IDX will choose the `simv` executable described in the configuration file. That is, the priority of IDX invoking `simv` is:

- `SIMV_DAIDIR` attribute in the configuration file > `simv` executable used in the command line.

If you specify the `simv` runtime options both in the command line and in the `SIMV_OPT` attribute in the configuration file, IDX will join all the runtime options together.

Using IDX in Verdi/VCS Interactive Debugging

Use the option `+idx_interactive_gui` to enable IDX waveform replay simulation in Verdi/VCS interactive debugging:

```
% simv -ucli +idx_cfg <IDX_configuration_file> \
+idx_work <IDX_working_dir> \
+idx_interactive_gui <other_simv_options>
```

In the Verdi/VCS interactive debugging, IDX does not exit and stop simulation until user exits Verdi or exits simulation by giving the UCLI command 'exit' through the Verdi console.

Checking IDX Results

Checking Files After IDX Simulation

After IDX waveform replay simulation, the resultant simulation logs are recorded at the working directory and the replayed waveform is recorded at the path where the `simv` executable locates. For example, consider the following `SIMV_OPT` attribute in the configuration file:

```
SIMV_DAIDIR = /tmp/user1/simv.daidir
SIMV_OPT = -l idx_sim.log +fsdbfile+replay.fsdb
```

After running the following IDX command:

```
% idx -cfg test.cfg -work IDX_WORK
```

The simulation log `idx_sim.log` is recorded to the directory `IDX_WORK` and the replayed FSDB `replay.fsdb` is generated at the path `/tmp/user1/`, which is the path that the `simv` executable locates.

Sanity Checks Between the Used Waveform and the IDX Results

Based on the signals mapping described in the mapping file, IDX intends to capture the values from the waveform and put the values to the corresponding signal in the `simv` during the simulation. To make sure IDX really applies the 'replay' process, the following command can be used for sanity checking by waveform comparison:

```
% idx -sanitychk <mapping_file> <replayed_waveform>  
<idx_used_waveform> [+time min max] [+x] [+z] [+maxerror num]  
[+sigmaxerror num]
```

Where,

- `<mapping_file>`: the mapping file described in [The Mapping File](#)
- `<replayed_waveform>`: the resultant waveform after IDX runtime replay simulation
- `<idx_used_waveform>`: the waveform described in the configuration file [The Mapping File](#)
- `[+time <min> <max>]`: the time window to do the waveform comparison
 - If not specified, default time window is the entire time window of `<replayed_waveform>`
 - Time unit is necessary for min and max time

For example:

```
+time 1000ps 150000ps
```

- `[+x]`: if specified, the 'x' value will be compared
- `[+z]`: if specified, the 'z' value will be compared
- `[+maxerror <num>]`: if specified, when the total mismatched number exceed the number `<num>`, the comparison process will be terminated
- `[+sigmaxerror <num>]`: if specified, when the mismatched number of a signal reach the number `<num>`, the comparison process will be terminated

Following is an example of sanity check command:

```
% idx -sanitychk replay.map replay.fsdb source.fsdb +x +z  
+maxerror 500
```

IDX Waveform Replay Simulation Configuration

Overview

The following are the valid attributes in the IDX waveform replay configuration file:

```
#####  
# Common fields  
# The field here will give impact on all waveform files  
#####  
  
//simv.daidir abs. path. [MUST HAVE] (Optional in simv. first usage)  
SIMV_DAIIDIR =  
  
//runtime options for running original simulation. (Optional)  
SIMV_OPT =  
  
//specify the path where the simv executable is going to run. (Optional)  
//default is the current directory if not specified  
SIMV_RUN_PATH =  
  
//waveform dumping Tcl commands. Use semicolon to separate  
tasks (Optional)  
WAVEFORM_DUMPING_TASK =  
  
//control the simv simulation time (Optional)  
SIMV_TIME_CONTROL =  
  
#####  
# Record fields (Waveform dedicated fields)  
# The field here will give impact to the waveform file for each record  
only #####  
  
##### record 1 #####  
{  
//waveform file abs. path. [MUST HAVE]  
WAVEFORM =  
  
//signal mapping list. [MUST HAVE]  
MAP=  
  
//the priority of the signal list when IDX put values to the simv.  
(Optional)  
KEY_MAP=  
  
//control the time window IDX put values to the simv. (Optional)  
REPLAY_TIME_DURATION=
```

```
//global time shift to control the time IDX put values to the
simv. (Optional)
DELAY=

//control the strength IDX puts values to the simv. (Optional).
//possible options [+force|+deposit|+force +release]. default: +force
MAP_REPLAY_TYPE=
}
```

Common Fields

The following table lists the common field attributes:

Attribute	Description
<code>SIMV_DAIDIR =</code>	[MUST HAVE] (Optional in <code>simv</code> . first usage) <code>simv.daidir</code> abs. path For example, <code>SIMV_DAIDIR = /tmp/guest/user1/simv.daidir</code>
<code>SIMV_OPT =</code>	(Optional) runtime options for running original simulation. For example, <code>SIMV_OPT = -l simv_eth.log</code> <code>+UVM_TESTNAME=eth_test +fsdbfile+idx_sim.fsdb</code>
<code>SIMV_RUN_PATH =</code>	(Optional) specify the path where the <code>simv</code> executable is going to run. Default is the current directory if not specified. For example, <code>SIMV_RUN_PATH = /tmp/user/work/</code>
<code>WAVEFORM_DUMPING_TASK =</code>	(Optional) waveform dumping Tcl commands. Use semicolon to separate tasks. For example, <code>WAVEFORM_DUMPING_TASK = fsdbDumpvars 0 +all;</code> <code>fsdbDumpSVA</code>

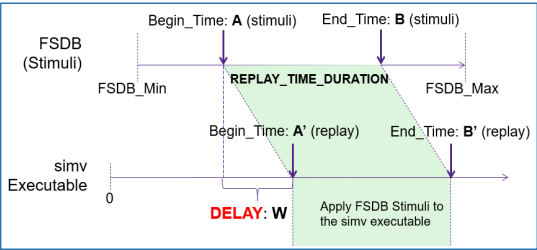
<code>SIMV_TIME_CONTROL =</code>	(Optional) control the <code>simv</code> simulation time. Syntax: <pre>SIMV_TIME_CONTROL = [+simv] [+stop=\$num[unit]] [+record=\$num[last] [, \$extend_num[unit]]]</pre> <code>+simv</code>: the simulation time is controlled by the <code>simv</code> executable. If <code>simv</code> was terminated at time T, IDX will stop at time T. If <code>simv</code> never stops when user gives 'run' in the UCLI prompt, then IDX never stops <code>+stop=\$num[unit]</code>: stop the simulation at a specific simulation time. If the <code>[unit]</code> is not specified, the <code>simv</code> resolution time unit will be used For example, <pre>SIMV_TIME_CONTROL = +simv SIMV_TIME_CONTROL = +stop=1550 SIMV_TIME_CONTROL = +stop=2550ps</pre>
----------------------------------	--

Record Fields

The following table lists the record field attributes:

Attribute	Description
<code>WAVEFORM =</code>	[MUST HAVE] waveform file abs. path. For example, <pre>WAVEFORM = /tmp/guest/user1/novas.fsdb</pre>
<code>MAP =</code>	[MUST HAVE] signal mapping list file. One signal mapping per line. Mapping File Format: <pre><simv_full_hier_name> => <waveform_full_hier_name> <simv_full_hier_name> => <const_value></pre> Example: <pre>MAP = /tmp/guest/user1/sig_map.txt</pre>

KEY_MAP =	(Optional) specify the signal list that has the priority to notify IDX when putting values to the <code>simv</code>. The <code>KEY_MAP</code> file is used to specify the clock/reset signals such that IDX can have clock/reset signals put values to the <code>simv</code> with higher order, and then put the values to the other signals. With this way the race condition in the simulation can be avoided. The signals described in the <code>KEY_MAP</code> file must be found in the <RHS> of the mapping file. That is, the signal in the <code>KEY_MAP</code> file is the signal in the waveform. Key_Map File Format: <waveform_sig_full_hier_name> [P = <num>]P = <num>: specify the priority of the signal to put value. The higher <num>, the higher priority. If not specified “P = <num>”, IDX treats the signal as “P = 1” Example: <pre>KEY_MAP = clk_rst.list</pre>
REPLAY_TIME_DURATION =	(Optional) specify the time window IDX put values to the <code>simv</code>. If not specified, IDX will use the time window of the waveform. Syntax: <pre>REPLAY_TIME_DURATION = ([<num>[unit]] - [<num>[unit]])[, ([<num>[unit]] - [<num>[unit]])]</pre> The [unit] can be one of the following words (s, ms, us, ns, ps, fs) and can be ignored; if not specified, the waveform time unit will be applied. The simple format is: <pre>(<BEGIN_TIME> - <END_TIME>)</pre> <BEGIN_TIME> can be ignored; then the minimal time in the waveform will be used. <END_TIME> can be ignored; then the maximum time in the waveform is used. Example: <pre>REPLAY_TIME_DURATION = (1100 - 5500) REPLAY_TIME_DURATION = (100ns - 300ns) REPLAY_TIME_DURATION = (0ns-200ns), (300ns-500ns), (700ns-900ns) REPLAY_TIME_DURATION = (- 6500ns) REPLAY_TIME_DURATION = (10ns -)</pre>

DELAY =	(Optional) global time shift to control the time IDX put values to the <code>simv</code>. The concept is as below picture illustrates.  With the setting of DELAY value, IDX will apply the stimulus to the <code>simv</code> since the time “A + W” to the time “B + W”. Note: The DELAY value ‘W’ can be positive or negative. If ‘A + W’ is smaller than 0, then time 0 is used in <code>simv</code> Example: DELAY = 100ns
MAP_REPLAY_TYPE =	(Optional) control the strength IDX puts values to the <code>simv</code>//possible options. default: <code>+force</code> Syntax: <pre>MAP_REPLAY_TYPE = [+force +deposit +release +force +release]</pre> If not specified <code>MAP_REPLAY_TYPE</code>, <code>+force</code> will be applied. When <code>MAP_REPLAY_TYPE</code> = <code>+force +release</code>, IDX will force the values since the begin time described in <code>REPLAY_TIME_DURATION</code>, and release those signals at the end time Example: <pre>MAP_REPLAY_TYPE = +force +release MAP_REPLAY_TYPE = +deposit</pre>

Examples of the Configuration File

The following is an example of the configuration file:

```
#####
# Common fields
# The field here will give impact on all waveform files
#####

SIMV_DAIDIR = /tmp/guest/user1/simv_ip.daidir
SIMV_OPT = -l simv_ip_replay.log +fsdbfile+ip_replay.fsdb
+ntb_random_seed=1234
```

```

WAVEFORM_DUMPING_TASK = fsdbDumpvars 0 +all
SIMV_TIME_CONTROL = +stop=15000ns

#####
# Record fields (Waveform dedicated fields)
# The field here will give impact to the waveform file for each record
only #####

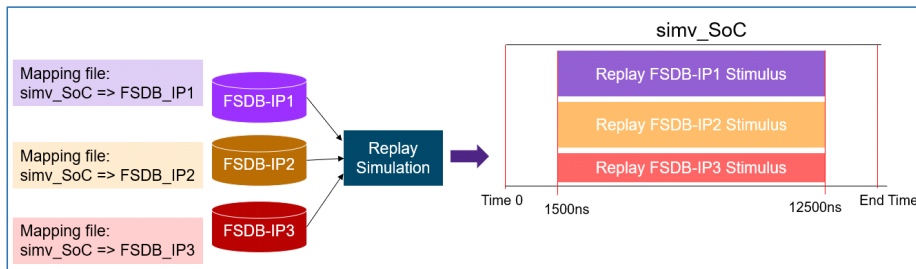
##### record 1 #####
{
WAVEFORM = /tmp/guest/deposit/novas.fsdb
MAP = $PWD/replay_signal.map
REPLAY_TIME_DURATION = ( - 29999ns)
MAP_REPLAY_TYPE = +force + release
}

```

IDX With Multiple Waveforms

Same or Overlapped Replayed Time Windows

The following figure and the configuration file illustrate how IDX apply multiple IP waveforms stimulus to the SoC `simv` executable with the same time window.



```

#####
# Common fields
# The field here will give impact on all waveform files
#####

SIMV_DAIDIR = /tmp/guest/user1/simv_soc.daidir
SIMV_OPT = -l simv_soc.log +fsdbfile+soc_replay.fsdb
WAVEFORM_DUMPING_TASK = fsdbDumpvars 0 +all

#####
# Record fields (Waveform dedicated fields)
# The field here will give impact to the waveform file for each record
only #####

```

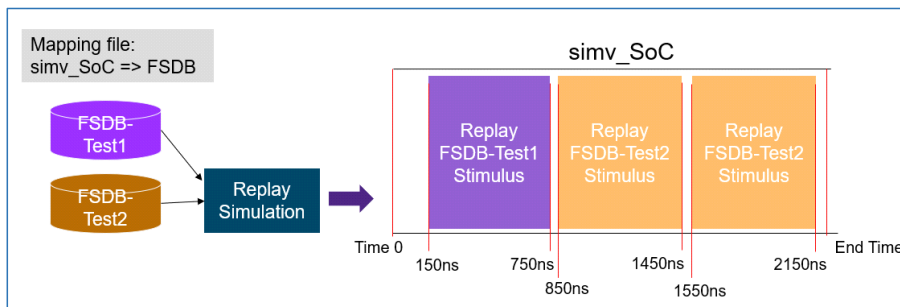
```
##### record 1 #####
{
WAVEFORM = /tmp/guest/deposit/ip1.fsdb
MAP = $PWD/soc_ip1.map
REPLAY_TIME_DURATION = (1500ns - 12500ns)
MAP_REPLAY_TYPE = +force + release
}

##### record 2 #####
{
WAVEFORM = /tmp/guest/deposit/ip2.fsdb
MAP = $PWD/soc_ip2.map
REPLAY_TIME_DURATION = (1500ns - 12500ns)
MAP_REPLAY_TYPE = +force + release
}

##### record 3 #####
{
WAVEFORM = /tmp/guest/deposit/ip3.fsdb
MAP = $PWD/soc_ip3.map
REPLAY_TIME_DURATION = (1500ns - 12500ns)
MAP_REPLAY_TYPE = +force + release
}
```

Non-overlapped Replayed Time Window

Below figure and the configuration file illustrate how IDX apply multiple test waveforms stimulus to the SoC `simv` executable with different time window. Note that the 2nd and the 3rd record fields in the configuration file are using the same FSDB but different time windows.



```
#####
# Common fields
# The field here will give impact on all waveform files
#####

SIMV_DAIDIR = /tmp/guest/user1/simv_soc.daidir
SIMV_OPT = -l simv_soc.log +fsdbfile+soc_replay.fsdb
```

```
WAVEFORM_DUMPING_TASK = fsdbDumpvars 0 +all

#####
# Record fields (Waveform dedicated fields)
# The field here will give impact to the waveform file for each record
  only
#####

##### record 1 #####
{
WAVEFORM = /tmp/guest/deposit/test1.fsdb
MAP = $PWD/soc_replay.map
REPLAY_TIME_DURATION = (150ns - 750ns)
}

##### record 2 #####
{
WAVEFORM = /tmp/guest/deposit/test2.fsdb
MAP = $PWD/soc_replay.map
REPLAY_TIME_DURATION = (850ns - 1450ns)
}

##### record 3 #####
{
WAVEFORM = /tmp/guest/deposit/test2.fsdb
MAP = $PWD/soc_replay.map
REPLAY_TIME_DURATION = (1550ns - 2150ns)
}
```

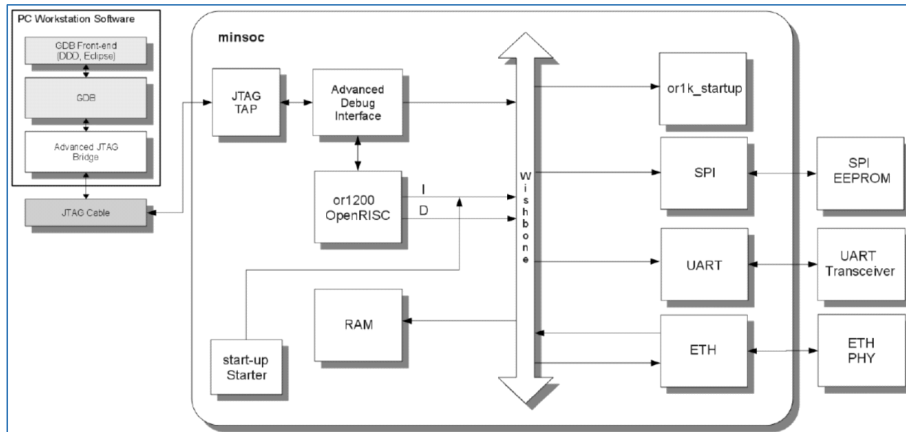
Executing an Example

The Design

You must use the MinSoC (Minimal OpenRISC System on Chip) design as an example to run IDX. The demo case is available in the following path:

```
$VERDI_HOME/demo/IDX/Waveform_Replay
```

The MinSoC architecture is illustrated below:



The entire design hierarchy including the Testbench is illustrated as follows:

Hierarchy	Module
minsoc_tb_top	minsoc_tb_top
minsoc_top_0	minsoc_top
clk_adjust	minsoc_clock_manager
dbg_top	adbg_top
ethmac	ethmac
ethreg1	eth_registers
maccontrol1	eth_maccontrol
macstatus1	eth_macstatus
miim1	eth_miim
rxethmac1	eth_rxethmac
txethmac1	eth_txethmac
wishbone	eth_wishbone
onchip_ram_top	minsoc_memory_model
or1200_top	or1200_top
tap_top	TAP_TOP(ARCH)
tc_top	minsoc_tc_top
uart_top	uart_top
minsoc_if	minsoc_io
init_fpga_memory	

The IP module we are going to demo the IDX is the Ethernet (ETH). The full hierarchical name in the SoC design is `minsoc_tb_top.minsoc_top_0.ethmac`.

SoC Waveform for Reuse

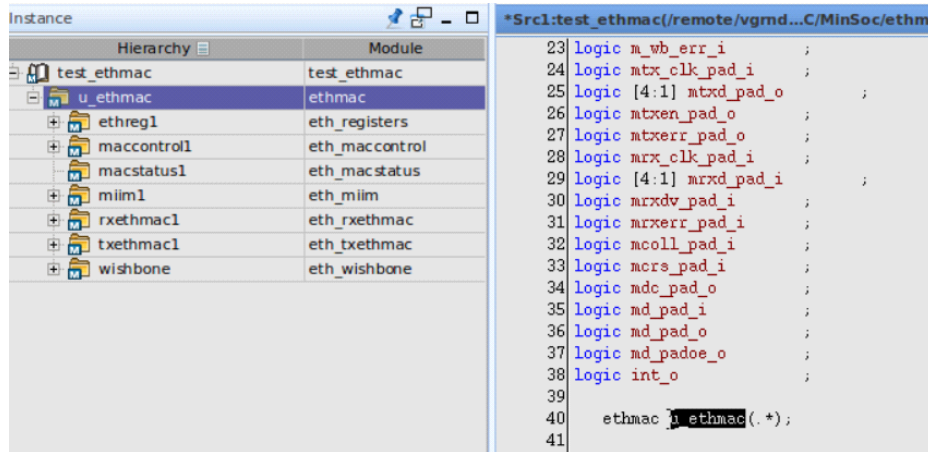
The `./minsoc/sim/soc.fsdb` is the FSDB to be reused to replay the IP “ethmac”.

Compiling the IP Design

Execute the following command to compile the “ethmac” IP design:

```
% make compile_ip
```

The `simv_ethmac` executable is generated in `./minsoc/ethmac_ip`. The design hierarchy of the IP design is shown as follows:



The `simv_ethmac` executable does not contain any stimulus to drive the IP “ethmac”. The Testbench module `test_ethmac` is just a wrapper to instantiate the IP.

Replying SoC Waveform Stimulus to the IP Design

Run the following command to perform IDX:

```
% make replay_block
```

The ‘replay_block’ tag in the Makefile describes the following commands:

```
% idx -cfg idx.cfg -work replay_ETHMAC_work
```

The configuration file `idx.cfg` used here is shown as follows:

```
#####
# Common fields
# The field here will give impact on all waveform files
#####

SIMV_DAIDIR = $ROOT/ethmac_ip/simv_ethmac.daidir

SIMV_OPT = -l ethmac.log +fsdbfile+ethmac_replay.fsdb

WAVEFORM_DUMPING_TASK = fsdbDumpvars +all;
```

Chapter 2: IDX Waveform Replay Simulation Usage Model

Executing an Example

```
#####
# Record fields (Waveform dedicated fields)
# The field here will give impact to the waveform file for each record
  only
#####

##### record 1 #####
{
WAVEFORM = $ROOT/sim/soc.fsdb

MAP = $PWD/ethmac_input.map
}
```

The mapping file `ethmac_input.map` is shown (partially) as follows:

```
test_ethmac.u_ethmac.wb_clk_i =>
  minsoc_tb_top.minsoc_top_0.ethmac.wb_clk_i
test_ethmac.u_ethmac.wb_rst_i =>
  minsoc_tb_top.minsoc_top_0.ethmac.wb_rst_i
test_ethmac.u_ethmac.wb_dat_i =>
  minsoc_tb_top.minsoc_top_0.ethmac.wb_dat_i
test_ethmac.u_ethmac.wb_adr_i =>
  minsoc_tb_top.minsoc_top_0.ethmac.wb_adr_i
test_ethmac.u_ethmac.wb_sel_i =>
  minsoc_tb_top.minsoc_top_0.ethmac.wb_sel_i
...
```

The scope hierarchy in the LHS is `test_ethmac.u_ethmac`, which is based on the `simv_ethmac` executable. The scope hierarchy in the RHS is `minsoc_tb_top.minsoc_top_0.ethmac`, which is based on the `soc.fsdb`. The mapping file contains all the inputs of the module “ethmac”.

After the IDX waveform replay simulation, the replayed FSDB `ethmac_replay.fsdb` and the simulation log file `ethmac.log` are generated.

Checking the Results

Since this example uses the same Ethernet design source codes for both the SoC design and the IP design, so after the IDX simulation, the outputs waveform should be the same between the two FSDB. Use below command to verify the fact.

```
% make compare_output_waveform
```

Or

```
% idx -sanitychk ./ethmac_output.map ./ethmac_replay.fsdb
$ROOT/sim/soc.fsdb
```


Where the mapping file `ethmac_output.map` is the mapping of the output ports of the module "ethmac". If there is no value mismatches, IDX gives the following message:

```
[IDX_INFO] No mismatch.
```

Environment Variables

The following table shows the environment variables that can be used when running IDX waveform replay simulation:

Environment Variable	Description
REP_PROF	When this environment variable is set to 1, the memory and runtime summary for all processes (<code>idx</code> , <code>simv</code> , <code>rep_wavereader</code>) invoked by IDX is dumped.
REP_WAVEREADER_HBNF_NOREPLAY_DISABLE	By default, if the <LHS> signal exists in the <code>simv</code> executable but the <RHS> signal does not exist in the waveform file, IDX gives ERROR messages and stops the runtime replay simulation. When this environment variable is set to 1, IDX prints the WARNING messages instead of stopping the process. By this way, you can identify which signals described in the mapping file do not exist in the waveform file.
REP_DISABLE_PERL_VERSION_CHECK	IDX requires the Perl version ≥ 5.006 to ensure that the functionality of the key map file is correct. By default, IDX checks the version if you specify the key map in the IDX configuration file. When this environment variable is set, IDX ignores the checking instead of stopping the process.
REP_VCS_INCR_COMPILE_EXECUTION_PATH	IDX requires VCS re-elaboration on a pure VHDL design to generate the new <code>simv</code> executable for waveform replaying. By default, IDX uses the same execution path of <code>simv.daidir</code> that you provide to do VCS re-elaboration. The execution path can be controlled by this environment variable.

REP_VCS_INCR_COMPILE_OUTPUT_PATH	IDX requires VCS re-elaboration on a pure VHDL design to generate the new <code>simv</code> executable for waveform replaying. By default, IDX puts a new <code>simv</code> executable and intermediate files under the hidden directory: <code>.snps_idx_vcs_incr_compile_dir</code>. The path of this hidden directory is under the path where IDX was invoked. The output path of hidden directory can be controlled by this environment variable.
REP_VCS_ENV_SYNOPSISYS_SIM_SETUP	IDX requires VCS re-elaboration on a pure VHDL design to generate the new <code>simv</code> executable for waveform replaying. By default, IDX refers to the <code>synopsys_sim.setup</code> specified in <code>simv.daidir</code> for re-elaboration. The path of <code>synopsys_sim.setup</code> for IDX reference can be controlled by this environment variable.
REP_VL_RECOG_CLK_DISABLE	By default, IDX utilizes the technique provided by VerdiLearn to recognize the clock signal list as the key map file on a Mixed-language/VHDL design to avoid potential race condition in VCS simulator. When this environment variable is set, IDX does not call VerdiLearn to recognize clock signals.
REP_VL_RECOG_CLK_FORCE	When this environment variable is set, IDX calls VerdiLearn to recognize the clock signal list as the key map file on a SystemVerilog/Verilog design.
REP_VL_RECOG_CLK_CONCAT_USER_KEY_MAP	By default, IDX does not touch the user-specified key map file. However, you can still ask IDX to append the recognized clock signals to the key map file by setting this environment variable.
REP_VL_CLK_PRIORITY	By default, the priority of the recognized clock signal is 2. The default priority can be changed by setting this environment variable.

Limitations and Known Issues

The limitations of IDX are listed as follows:

- **Mixed-language/VHDL support in IDX**
 - IDX needs VCS re-elaboration on a pure VHDL design to generate the new `simv` executable for waveform replaying. This may take a while but it needs to be done only once.
 - IDX cannot apply the stimulus to VHDL physical, time, string, access, and file data types in the `simv` executable.
 - IDX cannot apply the stimulus to VHDL variable in the `simv` executable.
 - Replaying the VHDL waveform to the SystemVerilog/Verilog design is not supported and vice versa. It might raise unexpected behavior due to the value system mismatch between VHDL and SystemVerilog/Verilog.
 - Potential race condition issues may arise on a Mixed-language/VHDL design in VCS simulator.
 - You must give proper priority for the signals to be replayed, that is, clock/reset signals, in the key map file to avoid a potential race condition problem.
 - If the priority is not specified, IDX attempts to recognize the clock signals and give them the priority. The default priority is 2. The default priority can be changed by setting the environment variable `REP_VL_CLK_PRIORITY`.
 - If the priority is specified, IDX does not touch the user-specified key map file. However, you still can ask IDX to append the recognized clock signals to the key map file by setting the environment variable `REP_VL_RECOG_CLK_CONCAT_USER_KEYMAP`.
- The IDX `simv` first usage on a pure VHDL design is not supported.
- The minimum version requirement of Perl for IDX is 5.006. The functionality of the key map file cannot work with the earlier versions of Perl.
- When using IDX in Verdi/VCS interactive debugging, the command “**restart**” is not supported.
- If the time precision is different between the `simv` executable and waveform, IDX will give the warning message like below (assume the time precision in `simv=ns`, and the `waveform=ps`).

```
[IDX_WARN-TPD] Time precision is diff. simv=-9 waveform(record0)=-12
```